LAPORAN TUGAS I

PENGEMBANGAN APLIKASI BERBASIS WEB (PABWE)

PEMBANGUNAN APLIKASI WEB *CALCORA — SMART WEB CALCULATOR* DENGAN MEMANFAATKAN *GENERATIVE ARTIFICIAL INTELLIGENCE* SEBAGAI ASISTEN PEMROGRAMAN



Nama	:	Dimas Rexan Samuel Sidabutar
NIM	:	11S24010
Mata Kuliah	:	Pengembangan Aplikasi Berbasis Web (PABWE)

PROGRAM STUDI INFORMATIKA

FAKULTAS INFORMATIKA DAN TEKNIK ELEKTRO

INSTITUT TEKNOLOGI DEL

TAHUN AJARAN 2026/2027

BAB I

PENDAHULUAN

1.1 Latar Belakang

Generative Artificial Intelligence (AI) kini banyak dipakai pengembang untuk merancang, menulis, menguji, dan memperbaiki kode. AI dapat menjelaskan konsep, membuat kerangka program, dan menyarankan perbaikan dengan cepat. Namun, kode dari AI tidak selalu benar. Hasilnya dapat memiliki kesalahan logika, tidak sesuai kebutuhan, atau kurang aman, sehingga pengembang tetap harus memberi instruksi yang jelas, memeriksa hasilnya, dan bertanggung jawab atas program akhir.

Pada Tugas I mata kuliah Pengembangan Aplikasi Berbasis Web (PABWE), program yang dibuat adalah kalkulator web bernama **Calcora Smart Web Calculator** dengan JavaScript. Kalkulator dipilih karena sederhana, tetapi tetap memuat hal penting: logika perhitungan, penanganan kesalahan, penyimpanan data, dan tampilan yang responsif. Hasil perhitungan juga mudah diuji karena jawabannya pasti (misalnya $2 + 3 \times 4 = 14$).

1.2 Tujuan

1. Membuat kalkulator web yang berfungsi: operasi dasar, persen, ubah tanda, riwayat tersimpan, dan tema terang/gelap.
2. Menggunakan AI sebagai asisten dengan *prompt* yang jelas, serta menjelaskan alasan di balik setiap *prompt*.
3. Mengevaluasi kode dari AI, mengidentifikasi kesalahan, memperbaikinya, dan menjelaskan keputusan yang diambil.
4. Menyimpan program dan laporan di repositori GitHub lengkap dengan cara menjalankan, inisialisasi, dan konfigurasinya.

BAB II
ANALISIS DAN PERANCANGAN

2.1 Analisis Kebutuhan

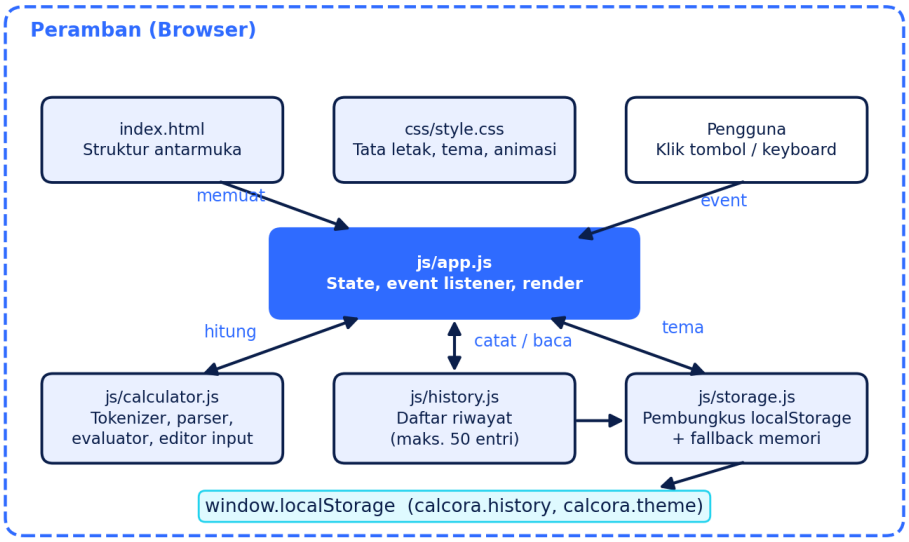
Tabel 2.1 Kebutuhan fungsional

Kode	Fitur	Deskripsi
FR-01	Operasi dasar	Tambah, kurang, kali, bagi dengan prioritas operator yang benar (× ÷ lebih dulu dari + −).
FR-02	Angka dan desimal	Tombol 0–9 dan titik desimal; titik ganda dan angka nol di depan (007) dicegah.
FR-03	Tombol =	Menghitung ekspresi dan menampilkan hasil.
FR-04	AC dan DEL	AC menghapus semua; DEL menghapus satu karakter terakhir.
FR-05	Persen dan ±	Persen membagi angka dengan 100; ± mengubah tanda angka terakhir.
FR-06	Masukan keyboard	Angka, + − * /, Enter, Backspace, Esc.
FR-07	Tampilan	Dua baris: ekspresi (kecil, dengan pratinjau) dan hasil (besar).
FR-08	Riwayat	Daftar perhitungan sebelumnya, disimpan di localStorage, dengan tombol hapus riwayat.
FR-09	Tema	Terang/gelap, pilihan disimpan di localStorage.
FR-10	Penanganan kesalahan	Bagi nol, ekspresi tidak valid, dan hasil tidak terdefinisi menampilkan pesan jelas.
FR-11	Aman	Ekspresi diproses parser sendiri; `eval()` tidak digunakan.
FR-12	Responsif	Tampilan rapi di laptop, tablet, dan ponsel.

Kebutuhan nonfungsional yang dijaga: hasil perhitungan konsisten (dibulatkan sesuai jumlah desimal operand dan maksimum 15 digit signifikan agar 0.1 + 0.2 tampil 0.3), kode dipisah per modul agar mudah dipelihara, aplikasi dapat dibuka langsung tanpa instalasi, dan tampilan mudah dipakai di layar kecil.

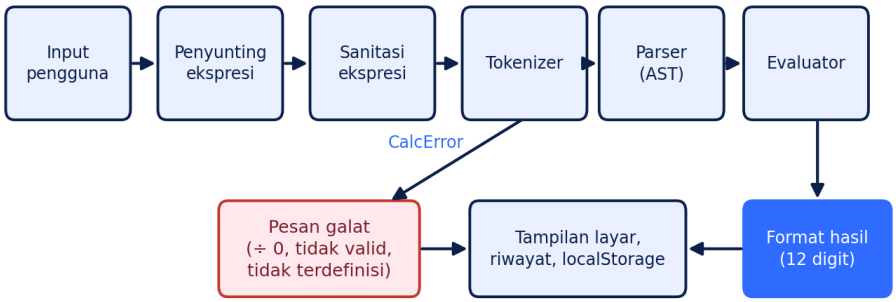
2.2 Rancangan Arsitektur

Aplikasi dibagi menjadi HTML (struktur), CSS (tampilan), dan empat modul JavaScript: `app.js` (penghubung antarmuka), `calculator.js` (mesin hitung), `history.js` (riwayat), dan `storage.js` (akses localStorage). Logika hitung dipisah dari tampilan agar dapat diuji sendiri.



Gambar 2.1 Arsitektur Calcora

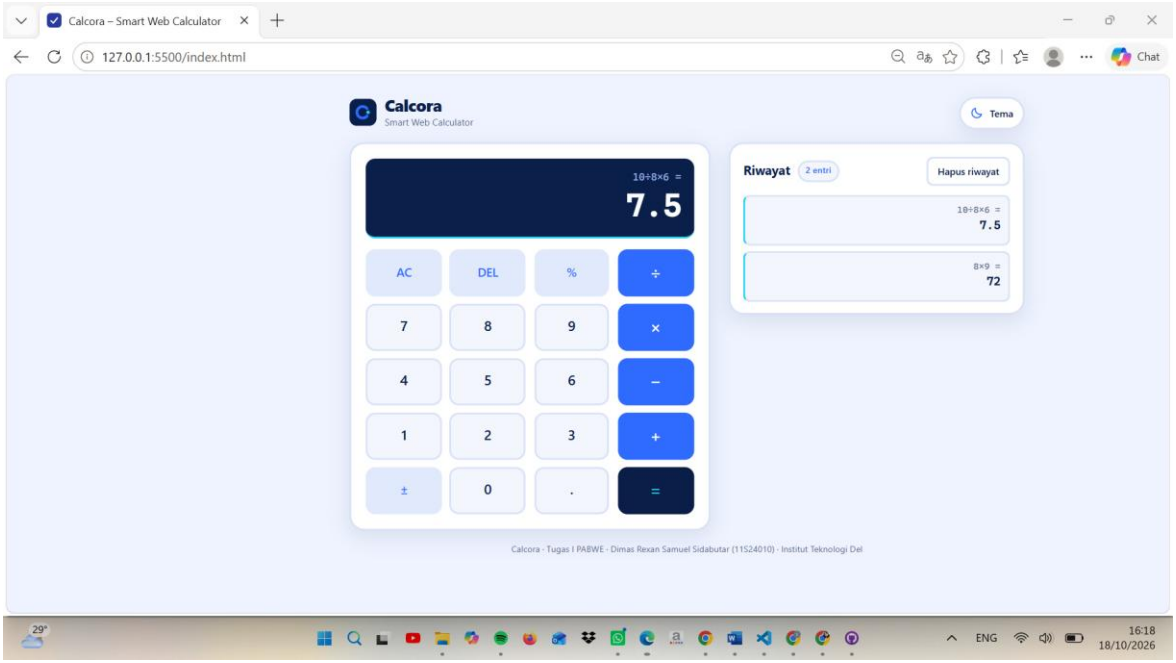
Saat tombol = ditekan, ekspresi dibersihkan, dipecah menjadi token, disusun menjadi pohon sintaks oleh *parser*, lalu dihitung. Hasil ditampilkan dan dicatat ke riwayat; bila gagal, pesan galat yang tampil.



Gambar 2.2 Alur perhitungan

2.3 Rancangan Antarmuka

Pada laptop, kalkulator berada di kiri dan riwayat di kanan; pada layar kecil riwayat pindah ke bawah. Warna yang dipakai: navy (#0B1F4B), biru elektrik (#2F6BFF), putih, dan aksen cyan (#22D3EE).



Gambar 2.3 Rancangan wireframe (gambar rancangan, bukan tangkapan layar)

2.4 Struktur Proyek

```
calcora-pabwe/
├── index.html          # Struktur antarmuka
├── css/style.css       # Tampilan, tema, responsif
├── js/
│   ├── storage.js      # Akses localStorage (+ cadangan memori)
│   ├── history.js      # Riwayat (maks. 50 entri)
│   ├── calculator.js   # Mesin hitung + penyunting input
│   └── app.js          # Penghubung UI dan modul
├── tests/
│   ├── calculator.test.js  # 82 kasus: mesin hitung
│   └── app.test.js         # 40 kasus: storage, riwayat, status app
├── docs/               # Laporan akhir (PDF)
├── package.json        # Skrip "npm test" (tanpa dependensi)
├── .gitignore
└── README.md           # Cara menjalankan, inisialisasi, konfigurasi
```

BAB III
PENULISAN KODE DENGAN BANTUAN AI

Kode ditulis bertahap dalam tiga tahap. Setiap tahap berisi *prompt* yang dikirim ke AI, **alasan** menyusun *prompt* tersebut, contoh hasil, lalu evaluasi dan keputusan saya. Pembagian bertahap dipilih agar hasil AI lebih mudah diperiksa dibandingkan meminta seluruh aplikasi sekaligus.

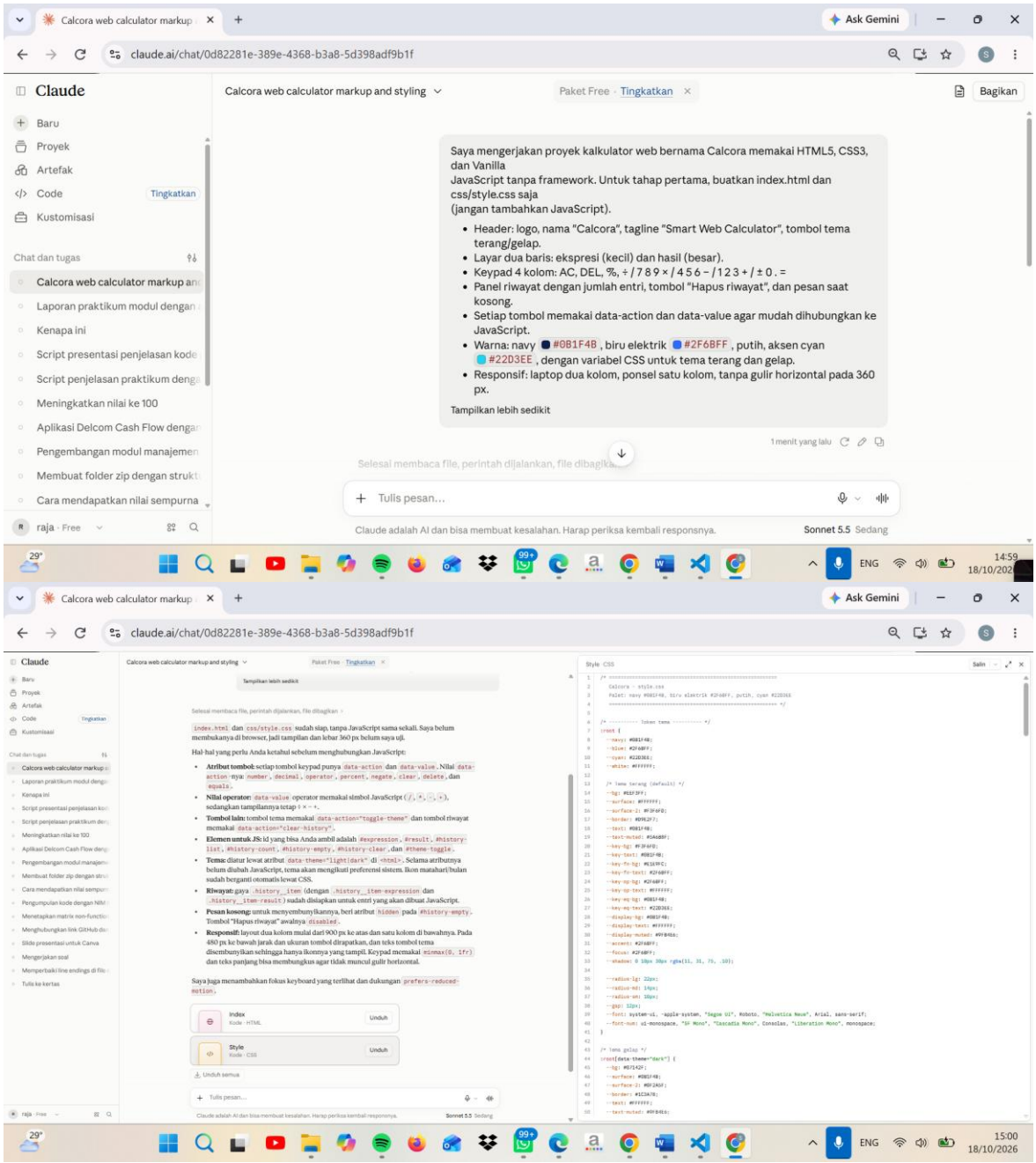
3.1 Tahap 1: Antarmuka (HTML dan CSS)

A. *Prompt* kepada AI

> Saya mengerjakan proyek kalkulator web bernama Calcora memakai HTML5, CSS3, dan Vanilla JavaScript tanpa framework. Untuk tahap pertama, buatkan index.html dan css/style.css saja (jangan tambahkan JavaScript).
>
> - Header: logo, nama "Calcora", tagline "Smart Web Calculator", tombol tema terang/gelap.
> - Layar dua baris: ekspresi (kecil) dan hasil (besar).
> - Keypad 4 kolom: AC, DEL, %, ÷ / 7 8 9 × / 4 5 6 − / 1 2 3 + / ± 0 . =
> - Panel riwayat dengan jumlah entri, tombol "Hapus riwayat", dan pesan saat kosong.
> - Setiap tombol memakai data-action dan data-value agar mudah dihubungkan ke JavaScript.
> - Warna: navy #0B1F4B, biru elektrik #2F6BFF, putih, aksen cyan #22D3EE, dengan variabel CSS untuk tema terang dan gelap.
> - Responsif: laptop dua kolom, ponsel satu kolom, tanpa gulir horizontal pada 360 px.

B. Alasan menyusun *prompt* seperti ini

- Dibatasi pada HTML dan CSS agar hasil tampilan dapat dinilai terpisah dari logika.
- Nama atribut (`data-action`, `data-value`) ditentukan sejak awal supaya tahap JavaScript tidak perlu mengubah HTML.
- Palet warna dan batas responsif disebut angka dan lebarnya, bukan "buat yang bagus", agar hasilnya terukur dan bisa diperiksa.



Gambar 3.1 Percakapan dengan AI pada Tahap 1 (prompt dan respons)

C. Contoh hasil kode

```
<div class="keypad" role="group" aria-label="Tombol kalkulator">
  <button type="button" class="key key--fn" data-action="clear" data-
value="AC">AC</button>
  <button type="button" class="key key--fn" data-action="delete" data-
value="DEL">DEL</button>
  <button type="button" class="key key--fn" data-action="percent" data-
value="%">%</button>
  <button type="button" class="key key--op" data-action="operator" data-value="/"
aria-label="Bagi">÷</button>
  ...
</div>
```

D. Evaluasi dan keputusan

Yang Diperiksa	Temuan Saya
Apakah memakai elemen semantik (header, main, aside, footer) dan setiap tombol punya data-action?	Ya. index.html memakai <header>, <main>, <section>, <aside>, dan <footer>; setiap tombol keypad memiliki data-action dan data-value. Semula tidak ada <footer>, sehingga saya menambahkannya (identitas pembuat) pada verifikasi akhir.
Apakah warna memakai variabel CSS dan teks terbaca di kedua tema?	Ya. Seluruh warna berada pada variabel di :root dan diganti pada tema gelap (data-theme="dark" dan prefers-color-scheme). Teks terbaca di kedua tema (lihat Gambar 4.3 dan 4.4).
Apakah ada @media dan tidak ada gulir horizontal pada 360 px?	Ya. Ada @media (min-width: 900px) untuk dua kolom dan @media (max-width: 480px) untuk ponsel; grid memakai minmax(0, 1fr). Pada lebar 360 px lebar dokumen sama dengan lebar layar (360 px), jadi tidak ada gulir horizontal (Gambar 4.6).
Keputusan yang saya ambil (bagian yang diubah, ditolak, atau dipertahankan) dan alasannya	Dipertahankan: struktur HTML, nama atribut data-*, dan palet warna karena sudah sesuai prompt. Ditambah: <footer> yang belum ada. Alasan: tahap berikutnya (JavaScript) bergantung pada data-action sehingga HTML tidak boleh berubah lagi.

3.2 Tahap 2: Logika Kalkulator

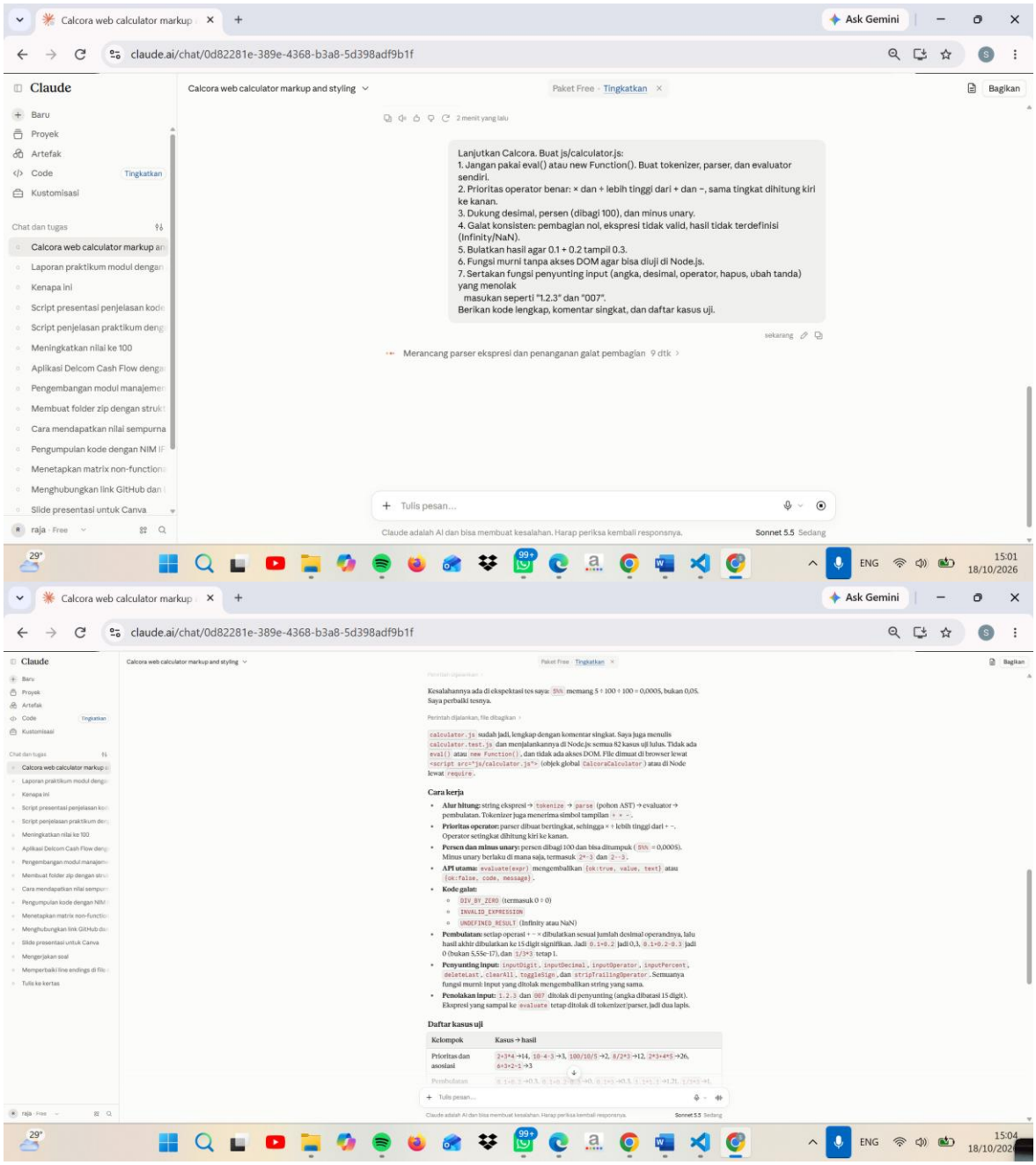
A. *Prompt* kepada AI

Lanjutkan Calcora. Buat js/calculator.js:
1. Jangan pakai eval() atau new Function(). Buat tokenizer, parser, dan evaluator sendiri.
2. Prioritas operator benar: × dan ÷ lebih tinggi dari + dan −, sama tingkat dihitung kiri ke kanan.
3. Dukung desimal, persen (dibagi 100), dan minus unary.
4. Galat konsisten: pembagian nol, ekspresi tidak valid, hasil tidak terdefinisi (Infinity/NaN).
5. Bulatkan hasil agar 0.1 + 0.2 tampil 0.3.
6. Fungsi murni tanpa akses DOM agar bisa diuji di Node.js.
7. Sertakan fungsi penyunting input (angka, desimal, operator, hapus, ubah tanda) yang menolak masukan seperti "1.2.3" dan "007".

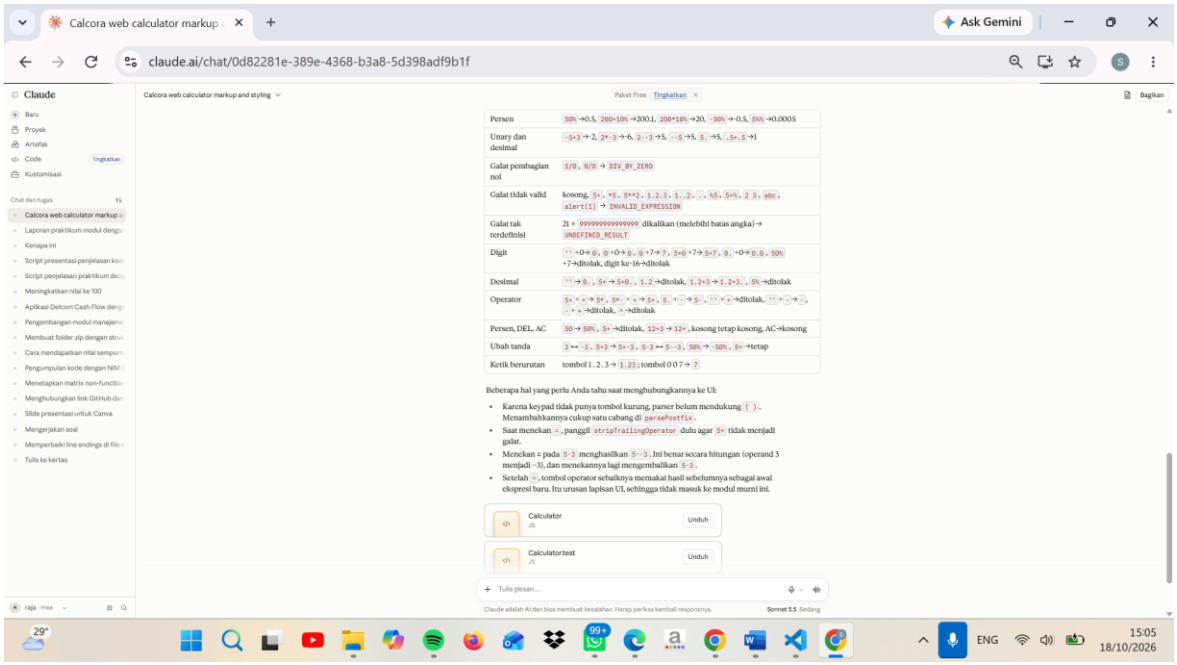
Berikan kode lengkap, komentar singkat, dan daftar kasus uji.

B. Alasan menyusun *prompt* seperti ini

- `eval()` dilarang secara eksplisit karena AI sering menjadikannya jalan pintas, padahal tidak aman untuk masukan pengguna.
- Aturan prioritas operator dan galat ditulis rinci karena itulah bagian yang paling sering keliru pada kalkulator.
- Meminta "fungsi murni tanpa DOM" dan "daftar kasus uji" agar hasil dapat diuji tanpa membuka peramban.


Calcora web calculator markup
Ask Gemini
claude.ai/chat/0d82281e-389e-4368-b3a8-5d398adf9b1f
Claude
Calcora web calculator markup and styling
Paket Free · Tingkatkan
Bagikan
Baru
Proyek
Artefak
Code
Tingkatkan
Kustomisasi
Chat dan tugas
Calcora web calculator markup an
Laporan praktikum modul dengan
Kenapa ini
Script presentasi penjelasan kode
Script penjelasan praktikum deng
Meningkatkan nilai ke 100
Aplikasi Delcom Cash Flow denga
Pengembangan modul manajemen
Membuat folder zip dengan struk
Cara mendapatkan nilai sempurna
Pengumpulan kode dengan NIM IF
Menetapkan matrix non-function
Menghubungkan link GitHub dan
Slide presentasi untuk Canva
raja · Free
2 menit yang lalu
Lanjutkan Calcora. Buat js/calculator.js:
1. Jangan pakai eval() atau new Function(). Buat tokenizer, parser, dan evaluator sendiri.
2. Prioritas operator benar: × dan ÷ lebih tinggi dari + dan −, sama tingkat dihitung kiri ke kanan.
3. Dukung desimal, persen (dibagi 100), dan minus unary.
4. Galat konsisten: pembagian nol, ekspresi tidak valid, hasil tidak terdefinisi (Infinity/NaN).
5. Bulatkan hasil agar 0.1 + 0.2 tampil 0.3.
6. Fungsi murni tanpa akses DOM agar bisa diuji di Node.js.
7. Sertakan fungsi penyunting input (angka, desimal, operator, hapus, ubah tanda) yang menolak
masukan seperti "1.2.3" dan "007".
Berikan kode lengkap, komentar singkat, dan daftar kasus uji.
sekarang
Merancang parser ekspresi dan penanganan galat pembagian 9 dtk
Tulis pesan...
Claude adalah AI dan bisa membuat kesalahan. Harap periksa kembali responsnya.
Sonnet 5.5 Sedang
29°
ENG
15:01
18/10/2026

Calcora web calculator markup
Ask Gemini
claude.ai/chat/0d82281e-389e-4368-b3a8-5d398adf9b1f
Claude
Kesalahannya ada di ekspektasi tes saya: 5% memang 5 ÷ 100 ÷ 100 = 0,0005, bukan 0,05. Saya perbaiki tesnya.
calculator.js sudah jadi, lengkap dengan komentar singkat. Saya juga menulis calculator.test.js dan menjalankannya di Node.js: semua 82 kasus uji lulus. Tidak ada eval() atau new Function(), dan tidak ada akses DOM. File dimuat di browser lewat <script src="js/calculator.js"> (objek global CalcoraCalculator) atau di Node lewat require.
Cara kerja
Alur hitung: string ekspresi → tokenize → parse (pohon AST) → evaluator → pembulatan. Tokenizer juga menerima simbol tampilan + × −.
Prioritas operator: parser dibuat bertingkat, sehingga × ÷ lebih tinggi dari + −. Operator setingkat dihitung kiri ke kanan.
Persen dan minus unary: persen dibagi 100 dan bisa ditumpuk (5% = 0,0005). Minus unary berlaku di mana saja, termasuk 2*-3 dan 2--3.
API utama: evaluate(expr) mengembalikan {ok:true, value, text} atau {ok:false, code, message}.
Kode galat:
DIV_BY_ZERO (termasuk 0 ÷ 0)
INVALID_EXPRESSION
UNDEFINED_RESULT (Infinity atau NaN)
Pembulatan: setiap operasi + − × dibulatkan sesuai jumlah desimal operandnya, lalu hasil akhir dibulatkan ke 15 digit signifikan. Jadi 0.1+0.2 jadi 0,3, 0.1+0.2-0.3 jadi 0 (bukan 5,55e-17), dan 1/3*3 tetap 1.
Penyunting input: inputDigit, inputDecimal, inputOperator, inputPercent, deleteLast, clearAll, toggleSign, dan stripTrailingOperator. Semuanya fungsi murni: input yang ditolak mengembalikan string yang sama.
Penolakan input: 1.2.3 dan 007 ditolak di penyunting (angka dibatasi 15 digit). Ekspresi yang sampai ke evaluate tetap ditolak di tokenizer/parser, jadi dua lapis.
Daftar kasus uji
Kelompok
Kasus → hasil
Prioritas dan asosiasi
2+3*4 → 14, 10-4-3 → 3, 100/10/5 → 2, 8/2*3 → 12, 2*3+4*5 → 26, 6+3+2-1 → 3
Pembulatan
Tulis pesan...
Claude adalah AI dan bisa membuat kesalahan. Harap periksa kembali responsnya.
Sonnet 5.5 Sedang
29°
ENG
15:04
18/10/2026



Gambar 3.2 Percakapan dengan AI pada Tahap 2 (logika kalkulator)

C. Cara kerja dan contoh kode

Ekspresi diproses tiga tahap: *tokenizer* memecah teks menjadi angka dan operator (karakter lain ditolak), *parser* menyusun pohon sintaks dengan urutan prioritas, dan evaluator menghitung nilainya. Prioritas ditentukan oleh urutan fungsi parser: `parseExpression` (+ −) memanggil `parseTerm` (× ÷), sehingga perkalian dan pembagian selalu terikat lebih dulu. Ekspresi 2 + 3 × 4 menjadi pohon `+(2, ×(3, 4))` bernilai 14.

```
function parseExpression() {
  let node = parseTerm();
  while (isOp(tokens[pos], '+-')) {
    const op = tokens[pos++].value;
    node = { type: 'binary', op, left: node, right: parseTerm() };
  }
  return node;
}

function parseTerm() {
  let node = parseUnary();
  while (isOp(tokens[pos], '*/')) {
    const op = tokens[pos++].value;
    node = { type: 'binary', op, left: node, right: parseUnary() };
  }
  return node;
}
```

Pembagian nol dan hasil tak hingga ditangani di evaluator sebelum hasil ditampilkan.

```
function evalNode(node) {
  switch (node.type) {
    case 'num':
      return node.value;
    case 'unary': {
      const v = evalNode(node.arg);
      return node.op === '-' ? -v : v;
```

```
    }
    case 'percent': {
      const v = evalNode(node.arg);
      return checkFinite(roundTo(v / 100, decimalsOf(v) + 2));
    }
    case 'binary': {
      const a = evalNode(node.left);
      const b = evalNode(node.right);
      const da = decimalsOf(a), db = decimalsOf(b);
      switch (node.op) {
        case '+': return checkFinite(roundTo(a + b, Math.max(da, db)));
        case '-': return checkFinite(roundTo(a - b, Math.max(da, db)));
        case '*': return checkFinite(roundTo(a * b, da + db));
        case '/':
          if (b === 0) fail('DIV_BY_ZERO');       // termasuk 0 ÷ 0
          return checkFinite(a / b);
      }
    }
  }
  return fail('INVALID_EXPRESSION');
}
```

D. Evaluasi dan keputusan

Yang Diperiksa	Temuan Saya
Apakah 2 + 3 × 4 menghasilkan 14 dan 10 − 6 ÷ 3 menghasilkan 8?	Ya. Diuji di browser: 2 + 3 × 4 = 14 dan 10 − 6 ÷ 3 = 8 (Tabel 4.1 nomor 3).
Apakah 10 ÷ 0 menampilkan pesan galat (bukan Infinity atau NaN)?	Ya. Layar menampilkan “Tidak bisa membagi dengan nol” (bukan Infinity/NaN) dan riwayat tidak bertambah. Pesan ini berbeda dari rancangan awal (“Tidak dapat dibagi nol”); saya menyesuaikan laporan dengan teks pada kode.
Apakah AI menggunakan eval() atau cara yang tidak aman? Cari kata "eval" dan "Function" pada kode.	Tidak. Pencarian “eval(” dan “new Function” pada folder js/ hanya menemukan fungsi evalNode (evaluator buatan sendiri) dan komentar yang menyatakan eval tidak dipakai. Tidak ada cara tidak aman.
Keputusan yang saya ambil (misalnya menolak eval, mengubah nama fungsi) dan alasannya	Dipertahankan: tokenizer–parser–evaluator buatan sendiri, karena sesuai larangan eval pada prompt. Dicatat: AI menambahkan pembulatan berbasis jumlah desimal (roundTo) di luar permintaan prompt; saya pertahankan karena 82 kasus uji membuktikan hasilnya benar (mis. 0.1 + 0.2 = 0.3).

3.3 Tahap 3: Penyimpanan, Riwayat, dan Integrasi

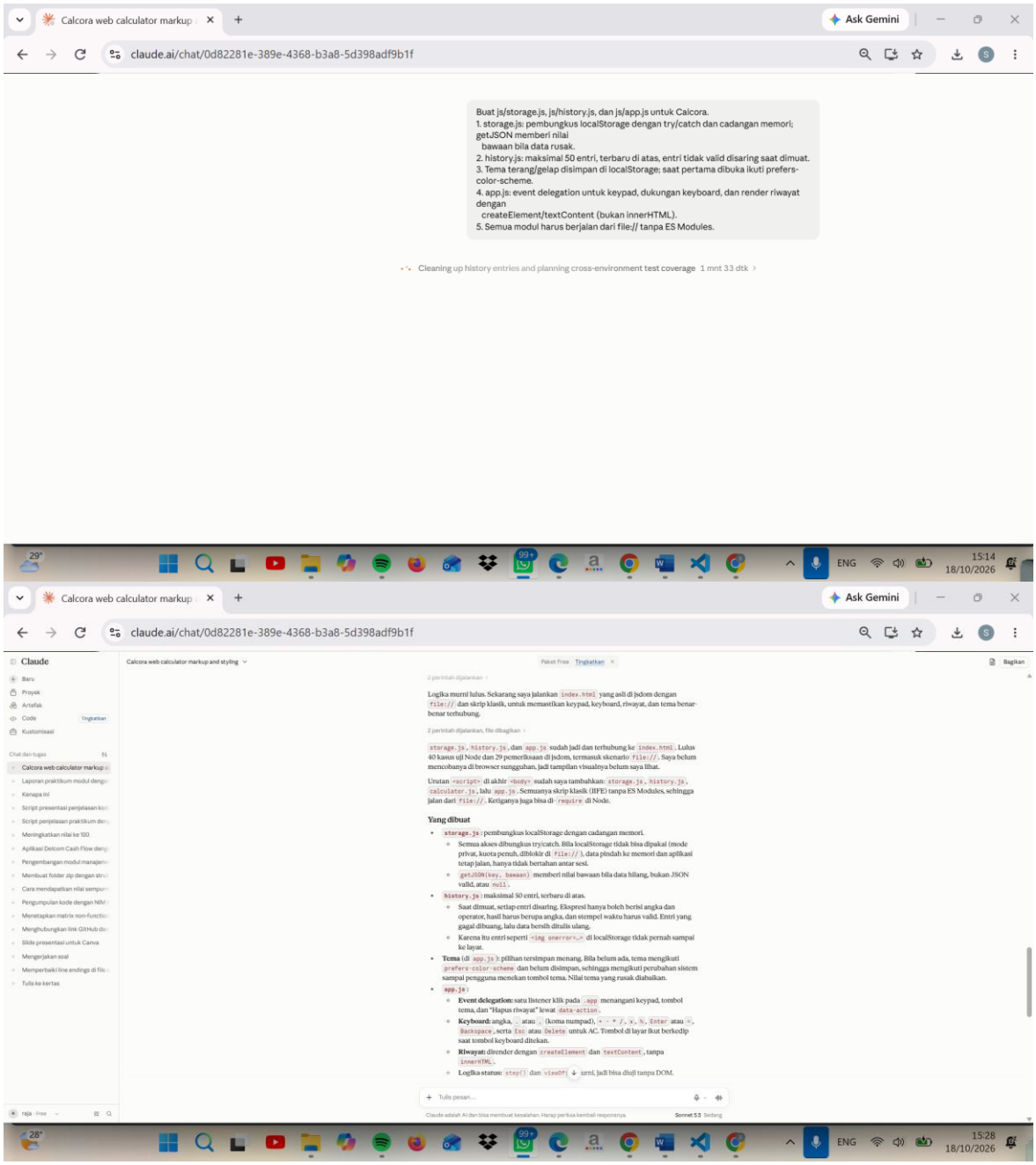
A. *Prompt* kepada AI

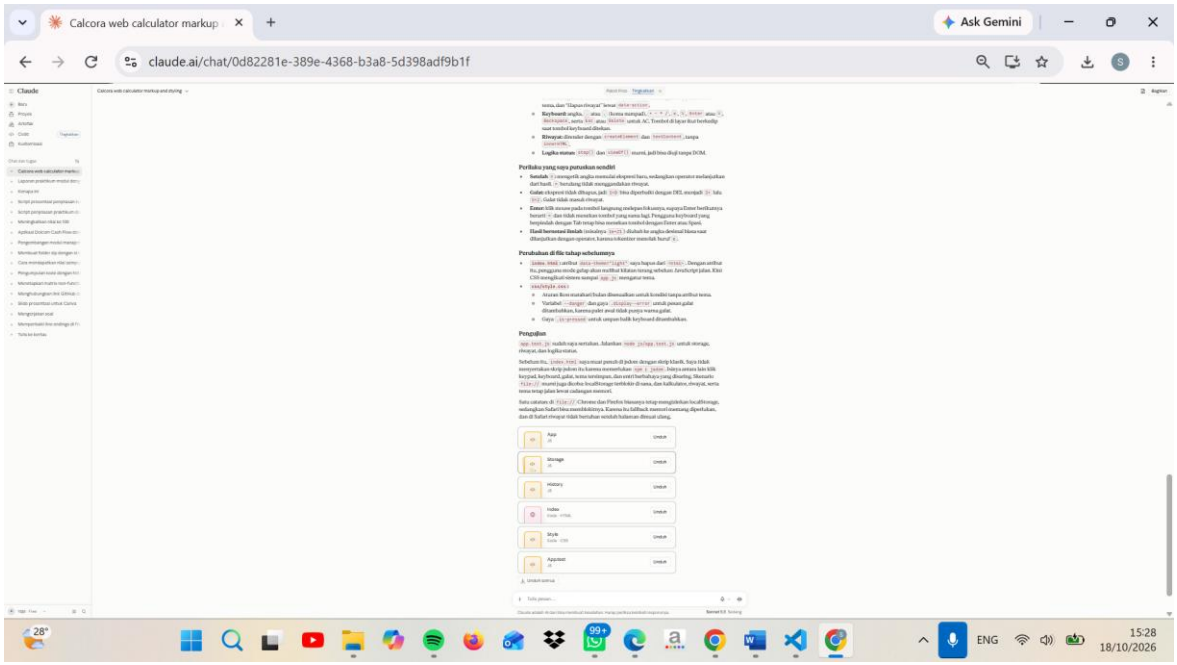
Buat js/storage.js, js/history.js, dan js/app.js untuk Calcora.
1. storage.js: pembungkus localStorage dengan try/catch dan cadangan memori; getJSON memberi nilai
   bawaan bila data rusak.
2. history.js: maksimal 50 entri, terbaru di atas, entri tidak valid disaring saat dimuat.
3. Tema terang/gelap disimpan di localStorage; saat pertama dibuka ikuti prefers-color-scheme.

4. app.js: event delegation untuk keypad, dukungan keyboard, dan render riwayat dengan createElement/textContent (bukan innerHTML).
5. Semua modul harus berjalan dari file:// tanpa ES Modules.

B. Alasan menyusun *prompt* seperti ini

- localStorage bisa gagal (mode privat, kuota penuh), jadi penanganan galat diminta sejak awal.
- `textContent` diminta karena `innerHTML` dengan data dari localStorage berisiko menyisipkan HTML.
- "Tanpa ES Modules" disebut karena `import` diblokir peramban bila halaman dibuka langsung dari berkas.


Calcora web calculator markup
Ask Gemini
claude.ai/chat/0d82281e-389e-4368-b3a8-5d398adf9b1f
Buat js/storage.js, js/history.js, dan js/app.js untuk Calcora.
1. storage.js: pembungkus localStorage dengan try/catch dan cadangan memori; getJSON memberi nilai
bawaan bila data rusak.
2. history.js: maksimal 50 entri, terbaru di atas, entri tidak valid disaring saat dimuat.
3. Tema terang/gelap disimpan di localStorage; saat pertama dibuka ikuti prefers-color-scheme.
4. app.js: event delegation untuk keypad, dukungan keyboard, dan render riwayat dengan
createElement/textContent (bukan innerHTML).
5. Semua modul harus berjalan dari file:// tanpa ES Modules.
Cleaning up history entries and planning cross-environment test coverage 1 mnt 33 dtk
ENG
15:14
18/10/2026
Calcora web calculator markup
Ask Gemini
claude.ai/chat/0d82281e-389e-4368-b3a8-5d398adf9b1f
Claude
Logika murni lulus. Sekarang saya jalankan index.html yang asli di jsdom dengan file:// dan skrip klasik, untuk memastikan keypad, keyboard, riwayat, dan tema benar-benar terhubung.
storage.js, history.js, dan app.js sudah jadi dan terhubung ke index.html. Lulus 40 kasus uji Node dan 29 pemeriksaan di jsdom, termasuk skenario file://. Saya belum mencobanya di browser sungguhan, jadi tampilan visualnya belum saya lihat.
Urutan <script> di akhir <body> sudah saya tambahkan: storage.js, history.js, calculator.js, lalu app.js. Semuanya skrip klasik (IIFE) tanpa ES Modules, sehingga jalan dari file://. Ketiganya juga bisa di-require di Node.
Yang dibuat
storage.js: pembungkus localStorage dengan cadangan memori.
Semua akses dibungkus try/catch. Bila localStorage tidak bisa dipakai (mode privat, kuota penuh, diblokir di file://), data pindah ke memori dan aplikasi tetap jalan, hanya tidak bertahan antar sesi.
getJSON(key, bawaan) memberi nilai bawaan bila data hilang, bukan JSON valid, atau null.
history.js: maksimal 50 entri, terbaru di atas.
Saat dimuat, setiap entri disaring. Ekspresi hanya boleh berisi angka dan operator, hasil harus berupa angka, dan stempel waktu harus valid. Entri yang gagal dibuang, lalu data bersih ditulis ulang.
Karena itu entri seperti <img onerror=…> di localStorage tidak pernah sampai ke layar.
Tema (di app.js): pilihan tersimpan menang. Bila belum ada, tema mengikuti prefers-color-scheme dan belum disimpan, sehingga mengikuti perubahan sistem sampai pengguna menekan tombol tema. Nilai tema yang rusak diabaikan.
app.js:
Event delegation: satu listener klik pada .app menangani keypad, tombol tema, dan "Hapus riwayat" lewat data-action.
Keyboard: angka, . atau , (koma numpad), + - * / , x , % , Enter atau =, Backspace, serta Esc atau Delete untuk AC. Tombol di layar ikut berkedip saat tombol keyboard ditekan.
Riwayat: dirender dengan createElement dan textContent, tanpa innerHTML.
Logika status: step() dan viewOf(↓ urnl, jadi bisa diuji tanpa DOM.
Tulis pesan…
Claude adalah AI dan bisa membuat kesalahan. Harap periksa kembali responsnya.
Sonnet 5.5
ENG
15:28
18/10/2026



Gambar 3.3 Percakapan dengan AI pada Tahap 3 (penyimpanan dan riwayat)

C. Contoh kode

Riwayat disimpan sebagai JSON setiap ada hasil baru, dan dibaca kembali saat halaman dimuat. Itulah yang membuat riwayat tetap ada setelah halaman di-*refresh*.

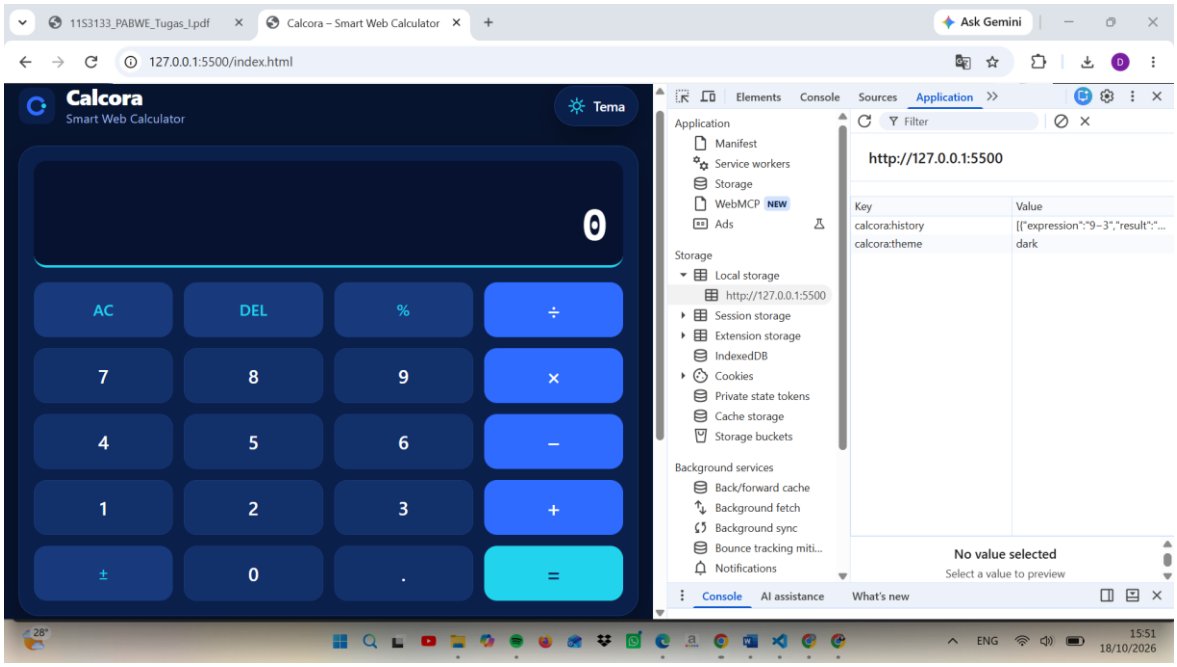
```
function sanitizeEntry(raw) {
  if (!raw || typeof raw !== 'object') return null;
  const { expression, result, timestamp } = raw;
  if (typeof expression !== 'string' || !EXPRESSION_RE.test(expression.trim()))
return null;
  if (typeof result !== 'string' || result.length > 40 || !RESULT_RE.test(result))
return null;
  if (typeof timestamp !== 'number' || !Number.isFinite(timestamp) || timestamp <
0) return null;
  return { expression: expression.trim(), result, timestamp };
}

function add(expression, result, now) {
  const entry = sanitizeEntry({ expression, result,
    timestamp: typeof now === 'number' ? now : Date.now() });
  if (!entry) return null;
  entries = [entry].concat(entries).slice(0, MAX_ENTRIES);   // terbaru di atas,
maks. 50
  persist();                                                 // simpan ke
localStorage
  return entry;
}
```

D. Evaluasi dan keputusan

Yang Diperiksa	Temuan Saya
Apakah riwayat tetap ada setelah F5 dan hilang setelah tombol hapus riwayat?	Ya. Setelah 2 perhitungan dan F5, riwayat tetap 2 entri; setelah klik “Hapus riwayat” menjadi 0 entri dan tombol nonaktif.

Yang Diperiksa	Temuan Saya
Apakah tema tersimpan dan diterapkan lagi saat halaman dibuka?	Ya. Setelah mengganti tema lalu F5, atribut data-theme tetap “dark”; nilainya tersimpan pada kunci calcora:theme.
Apakah aplikasi tetap jalan bila data localStorage diubah menjadi teks rusak?	Ya. Setelah calcora:history diisi teks bukan JSON (“{rusak”) lalu F5, aplikasi tetap berjalan dan riwayat tampil kosong (0 entri) tanpa galat pada Console.
Keputusan yang saya ambil dan alasannya	Dipertahankan: penyaringan entri (sanitizeEntry) dan cadangan memori pada storage.js. Diubah: kunci localStorage ternyata "calcora:history" dan "calcora:theme" (memakai awalan "calcora:"), bukan "calcora.history" seperti rancangan awal, sehingga laporan disesuaikan dengan kode.



Gambar 3.4 Bukti data di localStorage (DevTools)

BAB IV

PENGUJIAN DAN DEBUGGING

4.1 Pengujian Fungsional

Pengujian fungsional dijalankan pada peramban Chromium 141 yang dikendalikan Playwright (18 Oktober 2026). Setiap skenario pada Tabel 4.1 dijalankan dengan mengeklik tombol pada index.html yang dibuka langsung dari file://, lalu teks layar dibaca kembali. Hasil yang diharapkan ditetapkan sebelum pengujian.

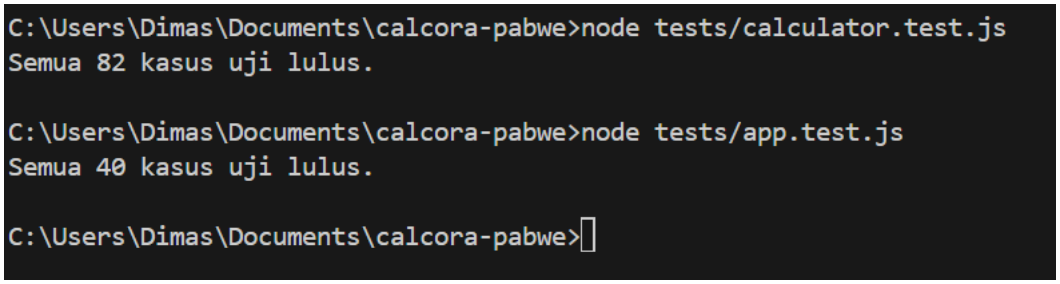
Tabel 4.1 Pengujian fungsional

No	Skenario	Input	Hasil Diharapkan	Hasil Aktual	Status
1	Penjumlahan	2 + 3 =	5	5	✓
2	Perkalian	12 × 4 =	48	48	✓
3	Prioritas operator	2 + 3 × 4 =	14	14 (dan 10 − 6 ÷ 3 = 8)	✓
4	Bagi nol	10 ÷ 0 =	Pesan "Tidak bisa membagi dengan nol"	“Tidak bisa membagi dengan nol”; riwayat tidak bertambah	✓
5	Desimal	0.1 + 0.2 =	0.3	0.3	✓
6	Desimal ganda	1 . 5 .	Tetap 1.5	1.5	✓
7	Hapus karakter	1 2 3 DEL	12	12	✓
8	Hapus semua	5 + 3 lalu AC	0	0	✓
9	Persen dan ubah tanda	5 0 % = ; 9 ±	0.5 ; −9	0.5 ; −9	✓
10	Keyboard	12*4 lalu Enter	48	48	✓
11	Riwayat tersimpan	hitung 2 ekspresi, F5	Riwayat tetap ada	2 entri tetap ada setelah F5	✓
12	Tema tersimpan	ganti tema, F5	Tema tetap	Tema gelap tetap setelah F5	✓
13	Responsif ponsel	lebar ±360 px	Tata letak utuh, tanpa gulir horizontal	Satu kolom; lebar dokumen 360 px = lebar layar (tanpa gulir horizontal)	✓

4.2 Pengujian Otomatis

Selain uji manual, tersedia dua berkas uji otomatis yang berjalan di Node.js tanpa pustaka tambahan: tests/calculator.test.js (82 kasus: mesin hitung dan penyunting input) dan tests/app.test.js (40 kasus: penyimpanan termasuk data rusak dan localStorage yang gagal,

riwayat, serta logika status aplikasi). Keduanya dijalankan dengan npm test. Hasil eksekusi: 122 lulus, 0 gagal (82 + 40).



Gambar 4.1 Hasil unit test di terminal

4.3 Identifikasi Kesalahan dan Debugging

Berikut lima potensi kesalahan yang relevan untuk kalkulator, masing-masing dengan cara menguji dan perbaikannya pada kode final. Kelimanya adalah potensi, bukan klaim bahwa kesalahan tersebut terjadi pada kode awal AI. Pada kode final, kelimanya sudah diuji dan hasilnya dicantumkan pada baris "Hasil setelah perbaikan". Kesalahan yang benar-benar ditemukan dicatat pada Tabel 4.3.

4.3.1 Kesalahan prioritas operator

Aspek	Uraian
Jenis	Potensi kesalahan (contoh yang relevan; belum tentu terjadi pada kode awal AI saya).
Gejala	2 + 3 × 4 menghasilkan 20, bukan 14.
Penyebab	Operator dihitung berurutan dari kiri ke kanan tanpa tingkat prioritas.
Cara menguji	Hitung 2 + 3 × 4 dan 10 − 6 ÷ 3; bandingkan dengan 14 dan 8.
Perbaikan pada kode final	Parser memisahkan tingkat prioritas: parseExpression (+ −) memanggil parseTerm (× ÷).
Hasil setelah perbaikan	2 + 3 × 4 = 14 dan 10 − 6 ÷ 3 = 8 (sesuai harapan). ☑ Berhasil ☐ Gagal ☐ Belum diuji

4.3.2 Pembagian dengan nol

Aspek	Uraian
Jenis	Potensi kesalahan (contoh yang relevan; belum tentu terjadi pada kode awal AI saya).
Gejala	Layar menampilkan Infinity atau NaN setelah 10 ÷ 0.
Penyebab	Di JavaScript 10 / 0 menghasilkan Infinity tanpa galat.
Cara menguji	Hitung 10 ÷ 0 dan 0 ÷ 0; pastikan muncul pesan galat dan riwayat tidak bertambah.
Perbaikan pada kode final	Evaluator memeriksa pembagi sebelum membagi dan melempar galat DIV_BY_ZERO; hasil tak hingga dilaporkan sebagai UNDEFINED_RESULT.
Hasil setelah perbaikan	10 ÷ 0 dan 0 ÷ 0 menampilkan "Tidak bisa membagi dengan nol"; riwayat tetap 2 entri. ☑ Berhasil ☐ Gagal ☐ Belum diuji

4.3.3 Input desimal ganda

Aspek	Uraian
Jenis	Potensi kesalahan (contoh yang relevan; belum tentu terjadi pada kode awal AI saya).
Gejala	Menekan titik berulang menghasilkan angka seperti 1.5.3.
Penyebab	Tombol titik menambah karakter tanpa memeriksa angka yang sedang diketik.
Cara menguji	Ketik 1 . 5 . dan pastikan tetap 1.5; ketik 5 + . dan pastikan menjadi 5 + 0.
Perbaikan pada kode final	`inputDecimal menolak titik kedua dan mengubah awal angka menjadi "0."; tokenizer/parser menolak angka salah bentuk.`
Hasil setelah perbaikan	Mengetik 1 . 5 . tetap 1.5 (sesuai harapan). ☑ Berhasil ☐ Gagal ☐ Belum diuji

4.3.4 Hasil tidak valid

Aspek	Uraian
Jenis	Potensi kesalahan (contoh yang relevan; belum tentu terjadi pada kode awal AI saya).
Gejala	0.1 + 0.2 tampil 0.30000000000000004, atau muncul "−0" dan Infinity.
Penyebab	Bilangan JavaScript berformat floating point biner; −0 dan Infinity tidak ditangani.
Cara menguji	Hitung 0.1 + 0.2, 1 ÷ 3, dan 0 × −1.
Perbaikan pada kode final	`roundTo membulatkan sesuai jumlah desimal operand, roundResult membatasi 15 digit signifikan dan mengganti −0 menjadi 0; hasil tak hingga menjadi galat UNDEFINED_RESULT.`
Hasil setelah perbaikan	0.1 + 0.2 = 0.3; 1 ÷ 3 = 0.333333333333333; hasil 0 tidak tampil sebagai −0. ☑ Berhasil ☐ Gagal ☐ Belum diuji

4.3.5 Riwayat tidak diperbarui atau layar kecil rusak

Aspek	Uraian
Jenis	Potensi kesalahan (contoh yang relevan; belum tentu terjadi pada kode awal AI saya).
Gejala	Riwayat tidak bertambah atau hilang setelah F5; pada 360 px tombol terpotong atau muncul gulir horizontal.
Penyebab	Daftar tidak dirender/disimpan setelah data berubah, JSON rusak tidak ditangkap; lebar elemen dikunci dalam piksel atau tanpa @media.
Cara menguji	Hitung dua ekspresi lalu F5; ubah calcora:history menjadi teks bukan JSON lalu F5; uji lebar 1280, 768, dan 360 px di mode perangkat DevTools.
Perbaikan pada kode final	`history.add langsung menyimpan ke localStorage; getJSON memberi nilai bawaan dan sanitizeList menyaring data rusak; grid memakai minmax(0, 1fr) dan @media mengubah tata letak ke satu kolom.`
Hasil setelah perbaikan	Riwayat tetap ada setelah F5; data calcora:history yang rusak tidak membuat aplikasi gagal; pada 360 px tidak ada gulir horizontal. ☑ Berhasil ☐ Gagal ☐ Belum diuji

Tabel 4.3 Kesalahan yang ditemukan pada verifikasi akhir kode dari AI

No	Kesalahan yang Ditemukan	Prompt Perbaikan ke AI	Hasil Setelah Perbaikan
1	tests/app.test.js gagal dijalankan dengan galat “Cannot find module ./storage.js” karena file berada di tests/ tetapi memanggil modul dengan jalur ./ (seharusnya ../js/). Komentar pada berkas juga menyebut jalur lama (node js/app.test.js).	Diperbaiki langsung pada verifikasi akhir (bukan lewat prompt): require diubah menjadi ../js/storage.js, ../js/history.js, ../js/app.js.	node tests/app.test.js: Semua 40 kasus uji lulus (Gambar 4.2).
2	Dokumentasi tidak sinkron dengan kode: README menyebut app.smoke.test.js dan jsdom (tidak ada), kunci localStorage ditulis "calcora.history" (kode: "calcora:history"), pesan galat bagi nol, nama konstanta (MAX_ITEMS, SIGNIFICANT_DIGITS), jumlah kasus uji (73, aktual 82 + 40), serta folder docs/ belum ada.	Diperbaiki langsung: README, laporan, dan tabel konfigurasi disesuaikan dengan kode; folder docs/ dan package.json (skrip npm test) ditambahkan.	Isi README dan laporan kini sama dengan perilaku kode yang diuji.

```
C:\Users\Dimas\Documents\calcora-pabwe>node tests/app.test.js
node:internal/modules/cjs/loader:1568
  throw err;
  ^

Error: Cannot find module './storage.js'
Require stack:
- C:\Users\Dimas\Documents\calcora-pabwe\tests\app.test.js
    at Module._resolveFilename (node:internal/modules/cjs/loader:1564:15)
    at wrapResolveFilename (node:internal/modules/cjs/loader:1118:27)
    at defaultResolveImplForCJSLoading (node:internal/modules/cjs/loader:1142:10
)
    at resolveForCJSWithHooks (node:internal/modules/cjs/loader:1169:12)
    at Module._load (node:internal/modules/cjs/loader:1341:5)
    at wrapModuleLoad (node:internal/modules/cjs/loader:261:19)
    at Module.require (node:internal/modules/cjs/loader:1674:12)
    at require (node:internal/modules/helpers:157:16)
    at Object.<anonymous> (C:\Users\Dimas\Documents\calcora-pabwe\tests\app.test
.js:3:27)
    at Module._compile (node:internal/modules/cjs/loader:1929:14) {
  code: 'MODULE_NOT_FOUND',
  requireStack: [ 'C:\\Users\\Dimas\\Documents\\calcora-pabwe\\tests\\app.test.j
s' ]
}

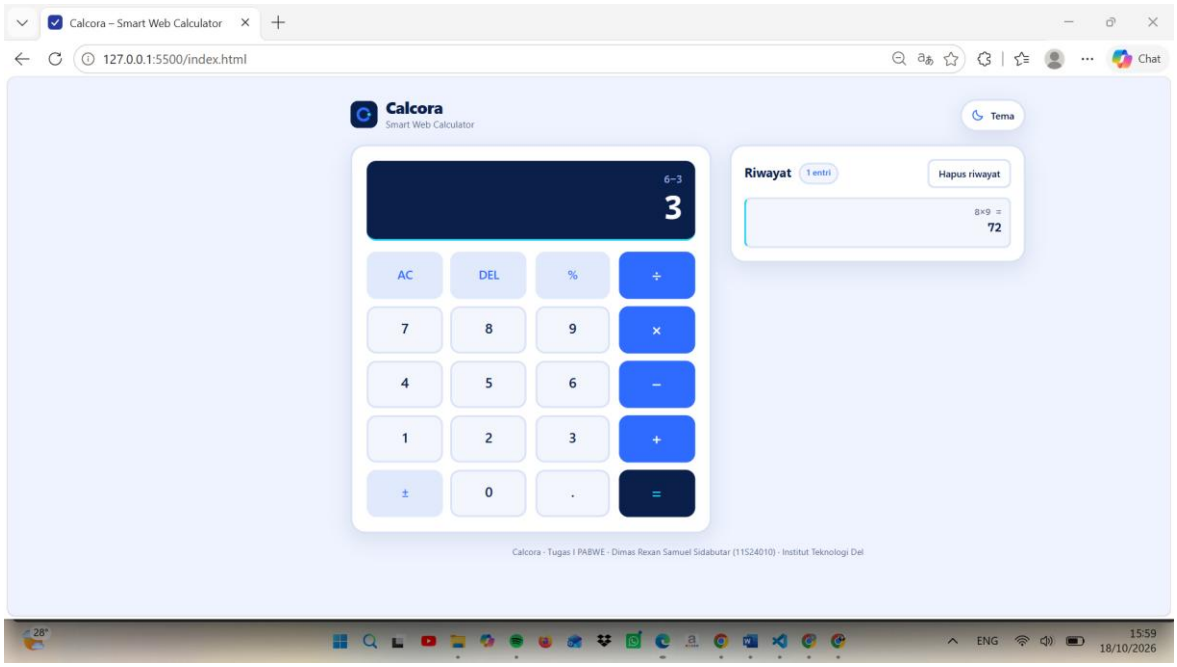
Node.js v24.21.0
```

```
C:\Users\Dimas\Documents\calcora-pabwe>node tests/app.test.js
Semua 40 kasus uji lulus.

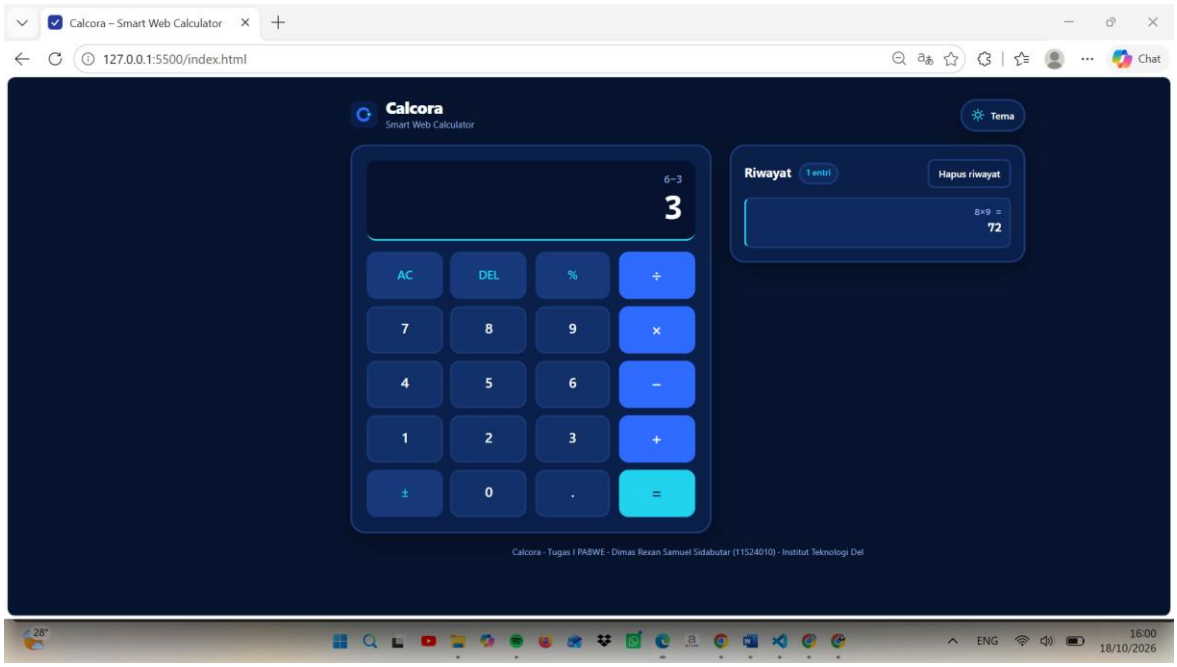
C:\Users\Dimas\Documents\calcora-pabwe>
```

Gambar 4.2 Bukti debugging: uji app.test.js gagal pada kode awal (atas) dan lulus setelah jalur require diperbaiki (bawah)

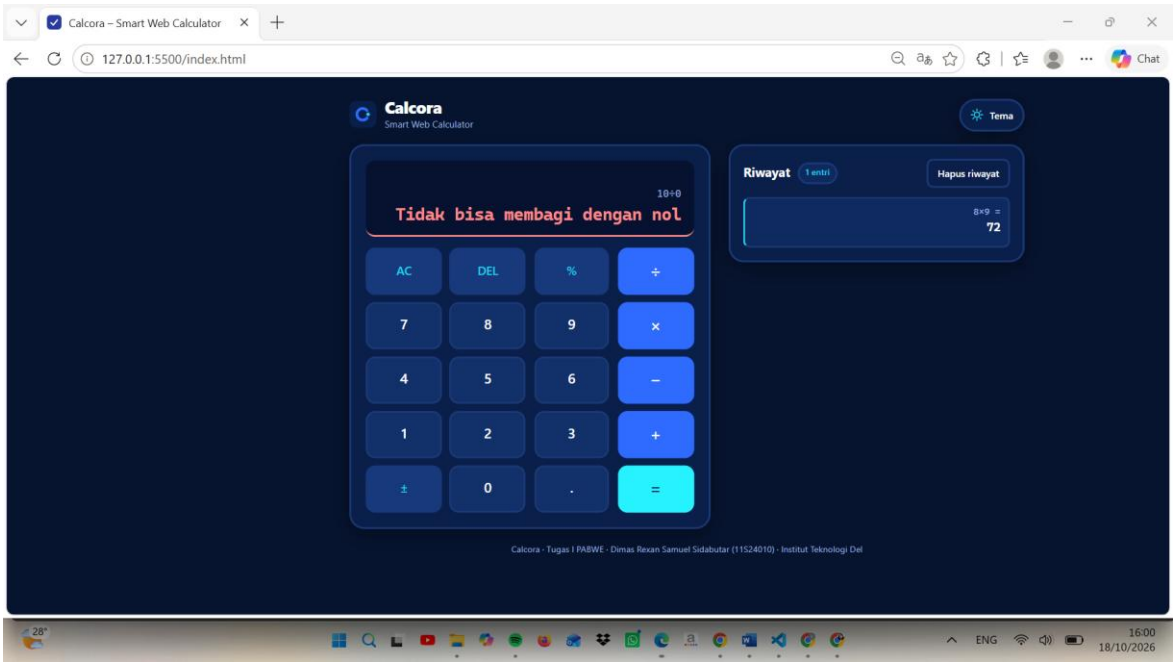
4.4 Hasil Akhir Aplikasi



Gambar 4.3 Tampilan Calcora mode terang



Gambar 4.4 Tampilan Calcora mode gelap



Gambar 4.5 Penanganan kesalahan 10 ÷ 0

BAB V

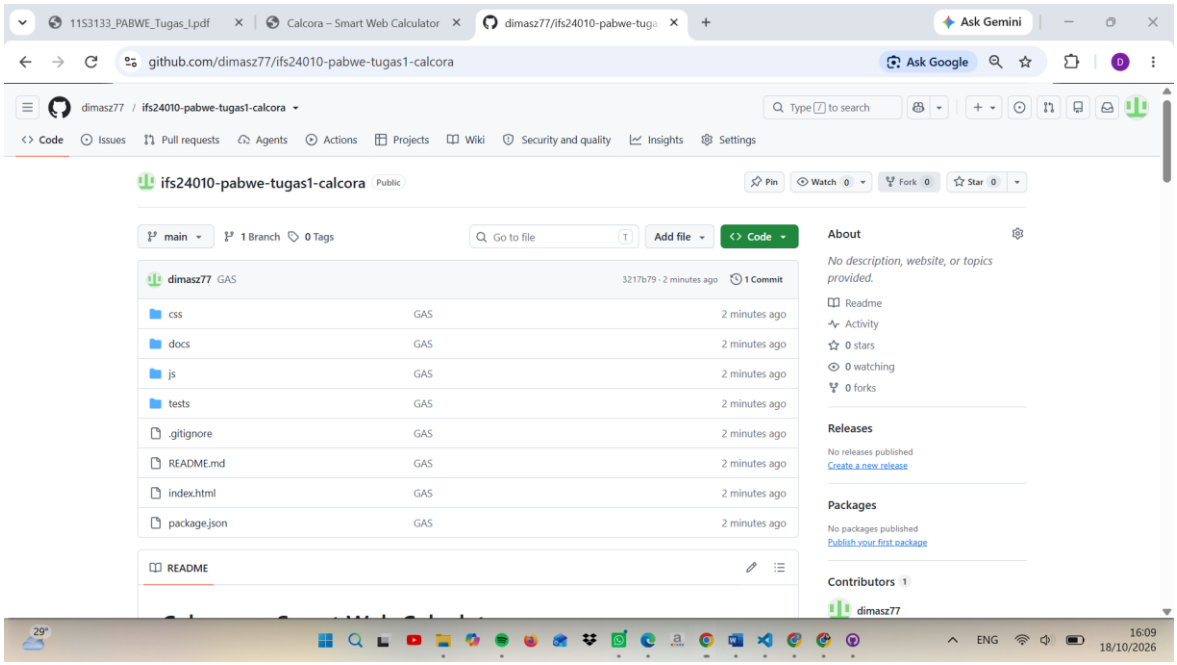
REPOSITORI GITHUB

5.1 Isi Repositori

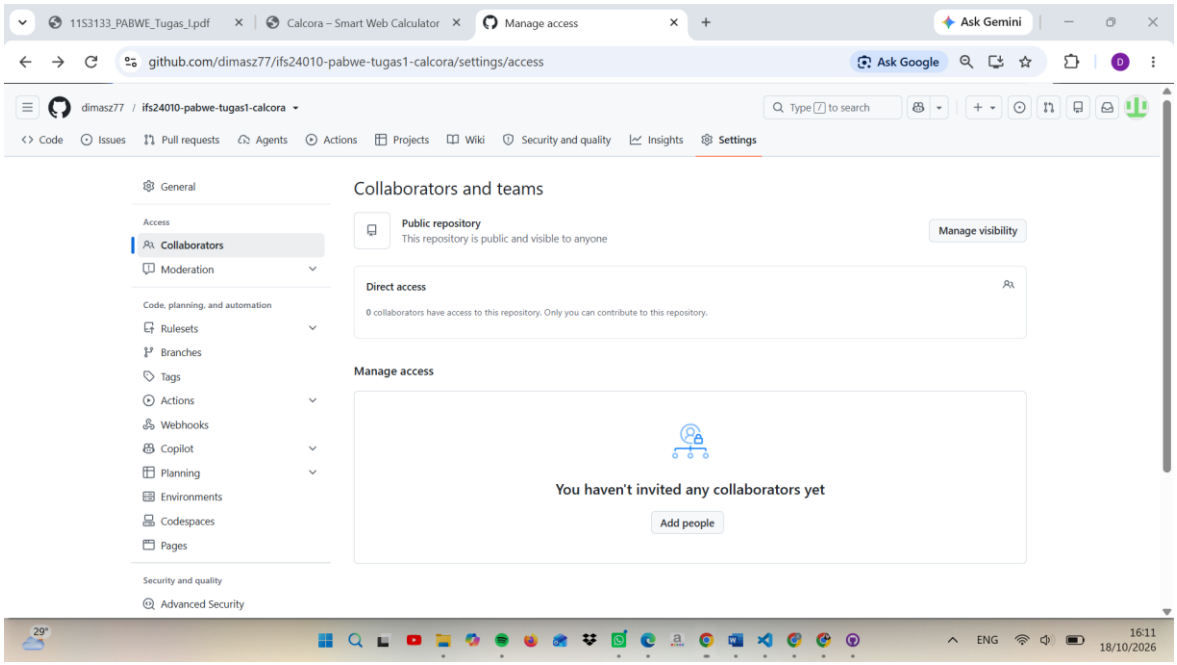
Repositori berisi kode program, README (cara menjalankan, inisialisasi, dan konfigurasi), unit test, serta laporan di folder `docs/`. Struktur berkas sama dengan Subbab 2.4.

Tabel 5.2 Informasi repositori (diisi setelah repositori dibuat)

Keterangan	Isi
Tautan repositori	https://github.com/dimasz77/ifs24010-pabwe-tugas1-calcora
Akun GitHub saya	dimasz77



Gambar 5.1 Halaman utama repositori GitHub



Gambar 5.2 Riwayat commit dan pengaturan kolaborator

BAB VI

PENUTUP

6.1 Kesimpulan

Calcora berhasil dibangun sebagai kalkulator web yang berfungsi dengan baik, mencakup operasi dasar, persen, ubah tanda, masukan keyboard, serta riwayat dan tema yang tersimpan di localStorage. Perhitungan dilakukan oleh *parser* buatan sendiri tanpa eval(), sehingga prioritas operator terjamin dan masukan pengguna lebih aman. Pengujian menunjukkan hasil yang sesuai harapan: seluruh 13 skenario uji manual berhasil, dan unit test menghasilkan 122 kasus lulus tanpa satu pun gagal (82 pada mesin hitung dan 40 pada penyimpanan, riwayat, dan logika aplikasi).

Selama pengerjaan, bantuan AI terbukti mempercepat penyusunan kode, tetapi hasilnya tidak dapat diterima begitu saja. Pada verifikasi akhir ditemukan satu kesalahan nyata pada kode dari AI, yaitu jalur *require* pada app.test.js yang salah sehingga berkas uji tidak dapat dijalankan, serta beberapa ketidaksesuaian antara dokumentasi dan kode, seperti nama kunci localStorage, pesan galat, dan jumlah kasus uji. Seluruhnya telah diperbaiki dan disesuaikan. Dengan demikian, penggunaan AI dalam pemrograman tetap menuntut instruksi yang jelas, pengujian yang teliti, dan tanggung jawab pengembang atas hasil akhir.

6.2 Refleksi

Tugas ini menunjukkan bahwa prompt yang jelas dan bertahap menghasilkan kode yang lebih mudah diperiksa. Selain itu, jawaban AI harus diperlakukan sebagai usulan, bukan kebenaran: kasus uji dengan jawaban yang sudah diketahui, seperti $2 + 3 \times 4 = 14$ dan $10 \div 0$ sebagai galat, membantu memastikan program benar. Verifikasi akhir juga menunjukkan bahwa kode yang berjalan baik belum tentu sesuai dengan dokumentasinya: berkas uji gagal dijalankan karena jalur yang salah, dan nama kunci, konstanta, serta pesan galat pada laporan berbeda dari kode. Karena itu setiap klaim pada laporan dicocokkan dengan menjalankan kode. Pemisahan logika dari tampilan juga membuat pengujian dan perbaikan lebih mudah.